Compiling Shor's Algorithm to the Dirac–Müller Quantum Tape: Linear Nearest-Neighbor Modular Exponentiation and Semiclassical Qubit Recycling in `|BF>`

mat-ocl (github.com/mat-ocl)
Systems & Quantum Esoteric Architecture Working Group
Technical Report · Architecture Specification v1.1 · October 2026

Abstract

Shor's polynomial-time integer factorization algorithm represents the foundational theoretical demonstration of quantum algorithmic supremacy over classical public-key cryptography. However, physical realization on early fault-tolerant lattices and Noisy Intermediate-Scale Quantum (NISQ) processors is severely constrained by 1D Linear Nearest-Neighbor (LNN) hardware topologies and prohibitive gate-synthesis overhead. In this work, we present a complete formal compilation of Shor's quantum order-finding algorithm into **`|BF>` (Bra-Ket-Fuck)**—a vacuum-initialized, 12-token Clifford+T quantum tape language operating on the Dirac–Müller Abstract Machine (DMM). We present an exact 6-qubit full-register program for factoring $N = 15$ with coprime base $a = 7$, showing that the modular multipliers reduce to networks of nearest-neighbor SWAPs (three transpositions, plus bit inversions for U_7), yielding an exact phase readout without truncation error. Furthermore, we develop a compact 5-qubit semiclassical iterative variant utilizing `|BF>`'s native active reset (`,`) and Copenhagen dynamic loop profile (`[ ]`), eliminating all two-qubit quantum Fourier transform phase rotations. We provide the complete formal syntax, mechanized statevector traces, quantum circuit schematics, an honest accounting of the cost of coherent control after lowering—including exact routing onto an open linear chain—and transpilation mappings targeting **OpenQASM 3.0** dynamic circuits.

1 Introduction & Motivation

The discovery by Peter Shor (1994, 1997) [1, 2] that integer factorization and discrete logarithms can be solved in bounded-error quantum polynomial time $\mathcal{O}((\log N)^2(\log\log N)(\log\log\log N))$ established quantum computing as a profound computational paradigm. At the heart of Shor's algorithm lies *Quantum Order Finding* (QOF), which recasts factorization into phase estimation of a unitary modular multiplication operator $U_a|y\rangle = |(a\cdot y) \pmod N\rangle$.

While textbook presentations assume an idealized execution model with arbitrary all-to-all qubit connectivity, physical quantum architectures (such as linear ion crystals, silicon spin qubits, and planar superconducting transmons) are fundamentally constrained by short-range physical interactions [7, 8]. In particular, 1D Linear Nearest-Neighbor (LNN) topologies mandate that two-qubit entangling gates act strictly between adjacent spatial sites (q_k, q_{k+1}).

Recently, the **`|BF>` (Bra-Ket-Fuck)** language and the **Dirac–Müller Abstract Machine (DMM)** [3] unified Urban Müller's minimal discrete tape semantics [4] with Dirac's Hilbert space bra-ket formalism [5, 6]. Operating over the physical vacuum state $|0\rangle^{\otimes N}$ with the universal Clifford+T set $\{\texttt{>}, \texttt{<}, \texttt{+}, \texttt{Z}, \texttt{H}, \texttt{T}, \texttt{-}, \texttt{\^{}}, \texttt{.}\}$, `|BF>` was shown to be BQP-universal, with its native two-qubit primitive restricted to nearest neighbors.

In this paper, we resolve two open questions in esoteric and verified quantum compilation:

1. How does a non-local modular exponentiation routine compile into a 1D tape head restricted to nearest-neighbor CNOT primitives (`^`)?

2. Can dynamic measurement semantics in `|BF>` achieve semiclassical qubit recycling (Kitaev 1995, Griffiths–Niu 1996) [9, 10] to execute Shor's algorithm on a strictly minimal register width?

2 The Dirac–Müller Abstract Machine

We review the formal operational semantics of `|BF>` as defined in [3]. A machine configuration is a tuple $\mathcal{M} = (\mathcal{H}_N, p, \mathcal{C})$, where $\mathcal{H}_N = (\mathbb{C}^2)^{\otimes N}$ is initialized

to the vacuum state:

$$|\Psi_0\rangle = \bigotimes_{k=0}^{N-1} |0\rangle_k = |00\ldots0\rangle \tag{1}$$

The classical pointer $p \in \mathbb{Z}_N$ designates the active tape cell ($p_0 = 0$), and $\mathcal{C} \in \{0,1\}^*$ denotes the classical measurement transcript stream.

Token	Operator / Action	Mathematical Definition
`>`	Shift Pointer Right	$p \leftarrow (p+1) \bmod N$
`<`	Shift Pointer Left	$p \leftarrow (p-1) \bmod N$
`+`	Pauli-X Bit-Flip	$\mathcal{U}_p(\|0\rangle\langle 1\| + \|1\rangle\langle 0\|)$
`Z`	Pauli-Z Phase-Flip	$\mathcal{U}_p(\|0\rangle\langle 0\| - \|1\rangle\langle 1\|)$
`H`	Hadamard Transform	$\mathcal{U}_p(\frac{1}{\sqrt{2}}\sum_{a,b}(-1)^{ab}\|a\rangle\langle b\|)$
`T`	$\pi/8$ Phase Gate	$\mathcal{U}_p(\|0\rangle\langle 0\| + e^{i\pi/4}\|1\rangle\langle 1\|)$
`-`	$T^\dagger$ Adjoint Phase	$\mathcal{U}_p(\|0\rangle\langle 0\| + e^{-i\pi/4}\|1\rangle\langle 1\|)$
`^`	Nearest-Neighbor CX	$\mathcal{U}_{p,p+1}(\|0\rangle\langle 0\| \otimes I + \|1\rangle\langle 1\| \otimes X)$
`.`	Projective Readout	Born collapse on q_p, appends $m \to \mathcal{C}$
`,`	Active Reset	Re-initializes cell p to $\|0\rangle$
`[ ]`	Coherent / Dynamic Loop	Profile C (Coherent) / Profile D (Dynamic)

Table 1: Primitive instruction set of the |BF> calculus.

2.1 Formal Loop Profiles

|BF> supports dual semantics for bracketed blocks `[` B `]`:

- **Profile C (Coherent Bounded Unitary):** When body B has net displacement $\Delta p(B) = 0$, contains no measurements or resets, leaves cell p invariant, and contains only Profile C admissible nested blocks, `[` B `]` compiles to a coherently controlled unitary conditioned on qubit p [11]:

$$\mathcal{C}_p(U_B) = \Pi_0^{(p)} \otimes I + \Pi_1^{(p)} \otimes U_B \tag{2}$$

  The control p interacts with every cell touched by B, so $\mathcal{C}_p(U_B)$ is in general *not* a nearest-neighbor operation: each `^` in B becomes a Toffoli gate. Section 7.2 quantifies the cost of routing such blocks onto a strict linear chain.

- **Profile D (Dynamic Copenhagen Loop):** Entry `[` executes a projective measurement $\Pi_m^{(p)}$ on the cell under the head. If $m = 1$, body B executes and control returns to `[` (re-measuring); if $m = 0$, control jumps past `]`. This enables mid-circuit measurement and classical feed-forward.

3 Quantum Order Finding for $N = 15$

To factor $N = 15$, we select a coprime integer $a \in \{2, 4, 7, 8, 11, 13\}$ such that $\gcd(a, N) = 1$. We adopt the canonical standard $a = 7$.

3.1 Subgroup Structure & Eigenstates

The sequence of powers $7^k \pmod{15}$ forms a cyclic subgroup $\langle 7\rangle \subset \mathbb{Z}_{15}^\times$ of order $r = 4$:

$$7^0 \equiv 1,\ 7^1 \equiv 7,\ 7^2 \equiv 4,\ 7^3 \equiv 13,\ 7^4 \equiv 1 \pmod{15} \tag{3}$$

The modular multiplication unitary $U_7|y\rangle = |(7y) \pmod{15}\rangle$ possesses four orthonormal eigenstates $|u_s\rangle$ for $s \in \{0, 1, 2, 3\}$:

$$|u_s\rangle = \frac{1}{2}\sum_{k=0}^{3} e^{-2\pi i \frac{sk}{4}} |7^k \pmod{15}\rangle \tag{4}$$

with eigenvalues $\lambda_s = e^{2\pi i(s/4)}$.

Because the sum of all eigenstates yields the computational state $|1\rangle$ (represented in LSB-first tape order $|y_0y_1y_2y_3\rangle$ where $y = \sum_{j=0}^{3} y_j 2^j$ as $|1000\rangle$):

$$\frac{1}{2}\sum_{s=0}^{3} |u_s\rangle = |1\rangle = |1000\rangle_{y_0y_1y_2y_3} \tag{5}$$

preparing the target register in $|1\rangle$ creates an exact uniform superposition across all eigencomponents. Since $r = 4 = 2^2$, the phase angles $\phi_s = s/4 \in \{0.00_2, 0.01_2, 0.10_2, 0.11_2\}$ require exactly $t = 2$ control qubits for zero-error phase estimation.

3.2 Classical Factor Extraction

Upon measuring phase $\phi = s/4$, continued fractions return the candidate order $r' = 4/\gcd(s, 4)$. For $s \in \{1, 3\}$ this gives $r' = 4$, which passes the verification $7^4 \equiv 1 \pmod{15}$, so $r = 4$. Since r is even and $7^{r/2} = 49 \equiv 4 \not\equiv -1 \pmod{15}$, non-trivial factors of 15 are extracted via Euclid's algorithm:

$$\gcd(7^{4/2} - 1, 15) = \gcd(48, 15) = 3 \tag{6}$$

$$\gcd(7^{4/2} + 1, 15) = \gcd(50, 15) = 5 \tag{7}$$

For $s = 2$ the reduced fraction $1/2$ yields $r' = 2$, which *fails* the verification ($7^2 \equiv 4 \not\equiv 1 \pmod{15}$) and is therefore not the order. The standard remedy when $\gcd(s, r) > 1$—testing small multiples of r'—finds $2r' = 4$ and recovers the factors. For $s = 0$ no information is obtained.

4 Linear Nearest-Neighbor Tape Compilation

4.1 Tape Allocation

The full-register quantum tape requires $N = 6$ cells: two control qubits (c_0, c_1) and four target qubits

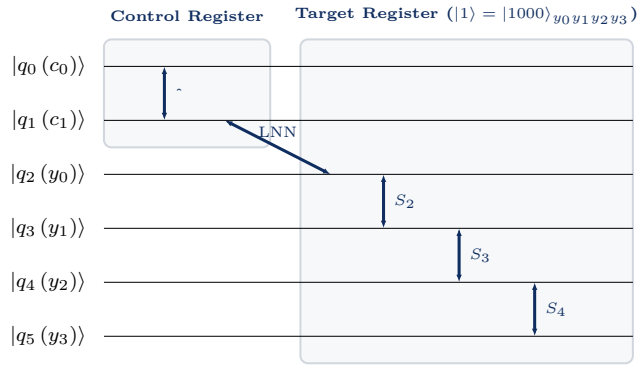


Figure 1: Linear Nearest-Neighbor (LNN) spatial tape topology in |BF>, illustrating the adjacent couplings used by the native ^ primitive. (Profile C control additionally couples the control cell to distant target cells.)

(y_0, y_1, y_2, y_3):

$$\text{Tape} = \Big[\underbrace{q_0}_{c_0}, \underbrace{q_1}_{c_1}, \underbrace{q_2}_{y_0}, \underbrace{q_3}_{y_1}, \underbrace{q_4}_{y_2}, \underbrace{q_5}_{y_3}\Big] \tag{8}$$

The pointer initializes at $p_0 = 0$ on cell c_0.

4.2 Nearest-Neighbor SWAP Synthesis

Because distant cells cannot interact directly, any non-local exchange requires a SWAP network. In |BF>, an adjacent SWAP operator $S_k \equiv \text{SWAP}_{k,k+1}$ is expressed as:

$$S_k = \texttt{\^{}H>H<\^{}>H<H\^{}} \tag{9}$$

This sequence executes 3 CNOTs and 4 pointer shifts, leaving pointer position invariant ($\Delta p = 0$).

4.3 Decomposition of Modular Permutations

Lemma 1 (Transposition Decomposition of U_4)**.** *On the 4-qubit target register, multiplication by* 4 (mod 15) *acts as a double transposition:*

$$U_4 \equiv SWAP(y_0, y_2) \cdot SWAP(y_1, y_3) \tag{10}$$

Compiled to LNN adjacent transpositions, this evaluates to:

$$U_4 = (S_2S_3S_2) \cdot (S_3S_4S_3) \tag{11}$$

Proof. Under modulo 15 multiplication by 4, the cyclic subgroup pairs exchange as: $1 \leftrightarrow 4$ and $7 \leftrightarrow 13$. In the LSB-first tape basis $|y_0y_1y_2y_3\rangle$:

$|1\rangle = |1000\rangle \leftrightarrow |4\rangle = |0010\rangle$ (transposes $y_0 \leftrightarrow y_2$)
$|7\rangle = |1110\rangle \leftrightarrow |13\rangle = |1011\rangle$ (transposes $y_1 \leftrightarrow y_3$)

Since $y_1 = y_3 = 0$ on $\{|1\rangle, |4\rangle\}$ and $y_0 = y_2 = 1$ on $\{|7\rangle, |13\rangle\}$, applying both transpositions independently preserves the invariant subspace. More generally, because $16 \equiv 1 \pmod{15}$, multiplication by $4 = 2^2$ is a cyclic rotation of the 4-bit word by two positions, so the identity holds on all inputs $y \in \{0, \ldots, 14\}$ (and fixes $y = 15$). Over an LNN chain, swapping sites j and $j+2$ decomposes into $S_jS_{j+1}S_j$. Thus, $\text{SWAP}(y_0, y_2) = S_2S_3S_2$ and $\text{SWAP}(y_1, y_3) = S_3S_4S_3$. □

Lemma 2 (Cyclic Shift Decomposition of U_7)**.** *On inputs $y \in \{1, \ldots, 14\}$ (in particular on the orbit $\{1, 7, 4, 13\}$), multiplication by* 7 (mod 15) *is equivalent to a sequential nearest-neighbor swap cascade followed by bit inversion:*

$$U_7 = (X_2 \otimes X_3 \otimes X_4 \otimes X_5) \cdot S_4 \cdot S_3 \cdot S_2 \tag{12}$$

executed left-to-right on the tape as S_2, then S_3, then S_4, followed by $X^{\otimes 4}$.

Proof. Tracing basis elements in $|y_0y_1y_2y_3\rangle$ under sequential application of S_2, S_3, S_4, and $X^{\otimes 4}$:
$|1\rangle = |1000\rangle \xrightarrow{S_2S_3S_4} |0001\rangle \xrightarrow{X^{\otimes 4}} |1110\rangle = |7\rangle$;
$|7\rangle = |1110\rangle \xrightarrow{S_2S_3S_4} |1101\rangle \xrightarrow{X^{\otimes 4}} |0010\rangle = |4\rangle$;
$|4\rangle = |0010\rangle \xrightarrow{S_2S_3S_4} |0100\rangle \xrightarrow{X^{\otimes 4}} |1011\rangle = |13\rangle$;
$|13\rangle = |1011\rangle \xrightarrow{S_2S_3S_4} |0111\rangle \xrightarrow{X^{\otimes 4}} |1000\rangle = |1\rangle$.
The permutation cycle $1 \to 7 \to 4 \to 13 \to 1$ is satisfied exactly, and all three SWAPs (S_2, S_3, S_4) act strictly on physical nearest neighbors. In general, $7y \equiv -8y \pmod{15}$: multiplication by 8 is a one-position cyclic rotation of the 4-bit word, and $15 - z$ is bitwise inversion, which proves the identity for every $y \in \{1, \ldots, 14\}$. (Outside this domain the circuit maps $|0\rangle \leftrightarrow |15\rangle$, which is irrelevant here because the target never leaves the orbit of $|1\rangle$.) □

5 The Complete Full-Register |BF> Program

Combining initialization, the controlled modular routines, and the 2-qubit inverse QFT yields Listing 1:

```
H>H>+<
[
  >^H>H<^>H<H^>^H>H<^>H<H^<^H>H<^>H<H^
  >^H>H<^>H<H^>^H>H<^>H<H^<^H>H<^>H<H^
  <<
]
<
[
  >>^H>H<^>H<H^>^H>H<^>H<H^>^H>H<^>H<H^
```

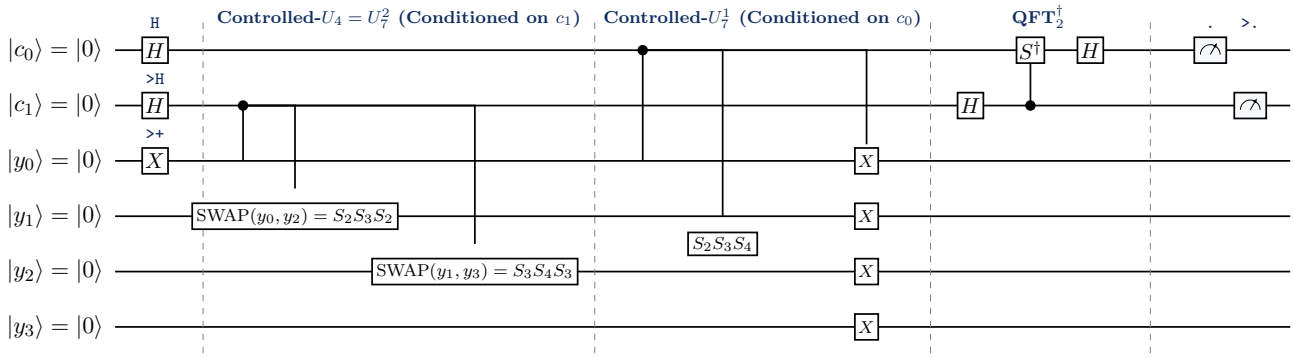


Figure 2: Compiled quantum circuit representation of Shor's Order-Finding algorithm ($N = 15, a = 7$) on a 6-qubit LNN tape.

```
  <<+>+>+>+
  <<<<<
]
>H[<->]<H
.>.
```

Listing 1: Complete |BF> program for Shor's Algorithm ($N = 15, a = 7$).

5.1 Line-by-Line Tape Verification

1. **Initialization (H>H>+<):** Pointer begins at $p = 0$. Applies H on c_0, shifts to c_1 ('>'), applies H, shifts to y_0 ('>'), applies Pauli-X ('+') to prepare $|1\rangle = |1000\rangle_{y_0y_1y_2y_3}$, and steps left to c_1 ('<') at $p = 1$.

2. **Controlled-U_4 Conditioned on c_1:** Enters '[' at $p = 1$. The block steps through $y_0 \leftrightarrow y_2$ via >S2>S3<S2 and $y_1 \leftrightarrow y_3$ via >S3>S4<S3, ending with « to return pointer to $p = 1$. Pointer displacement $\Delta p = 0$; c_1 is untouched. Profile C executes Controlled-U_4.

3. **Controlled-U_7 Conditioned on c_0:** Operator '<' positions pointer at c_0 ($p = 0$). Enters '['. Executes cyclic shift »S2>S3>S4, steps back via «, applies Pauli-X across all four cells via +>+>+>+, and traverses left five cells via «««< back to $p = 0$. Profile C executes Controlled-U_7.

4. **Adjacent Inverse QFT ($\mathbf{QFT}_2^\dagger$):** At $p = 0$, shifts to c_1 ('>') and applies H. Macro [<->] implements controlled-$S^\dagger$: conditioned on $c_1 = 1$, moves left to c_0, executes $S^\dagger = T^\dagger T^\dagger \equiv$ -, and returns right. Steps left to c_0 ('<') and applies H.

5. **Readout (.>.):** Measures $c_0 \to \phi_1$, shifts to c_1, and measures $c_1 \to \phi_2$, terminating execution.

5.2 Statevector Evolution & Sampling

Tracing the 64-dimensional composite statevector $|\Psi(t)\rangle$ through execution confirms that all cross-terms interfere constructively into exact basis states:

$$|\Psi_{\text{final}}\rangle = \frac{1}{2}\sum_{s=0}^{3} |s\rangle_{c_0c_1} \otimes |u_s\rangle_{y_0y_1y_2y_3} \tag{13}$$

Sampling the classical stream $\mathcal{C} = (c_0, c_1)$ generates the exact probability distribution in Table 2.

Bitstring (c_0c_1)	Phase ϕ	Probability	Order r	Extracted Factors
0 0	$0.00_2 = 0$	25.0%	Trivial ($s = 0$)	*(Resample)*
0 1	$0.01_2 = 1/4$	25.0%	$r = 4$	**3, 5**
1 0	$0.10_2 = 1/2$	25.0%	$r' = 2$ (fails check) $\to 2r' = 4$	**3, 5** (after multiple test)
1 1	$0.11_2 = 3/4$	25.0%	$r = 4$	**3, 5**

Table 2: Measurement statistics and factor extraction for $N = 15$ (full register: c_0 is the most significant phase bit).

The order is read off directly by continued fractions with probability $\varphi(4)/4 = 50\%$ ($s \in \{1, 3\}$); including the multiple test for $s = 2$, the single-shot success probability of the classical post-processing is 75%.

6 Semiclassical Recycling: 5-Qubit Implementation

While the full-register approach requires 6 qubits, Kitaev's semiclassical iterative phase estimation (IPE) [9, 10, 12] demonstrates that control qubits can be measured sequentially from least significant bit (LSB) to most significant bit (MSB), allowing a single control qubit to be recycled across iterations.

6.1 Dynamic Loop Architecture

In |BF>, semiclassical recycling utilizes the active reset token , and **Profile D** dynamic loops to im-

plement classical feed-forward corrections:

```
>+<
H [ >^H>H<^>H<H^>^H>H<^>H<H^<^H>H<^>H<H^
    >^H>H<^>H<H^>^H>H<^>H<H^<^H>H<^>H<H^ <<
     ] H .
[ >>>>+<<<< , ]
H >>>> [ <<<<-->>>> , ] <<<<
[ >^H>H<^>H<H^>^H>H<^>H<H^>^H>H<^>H<H^
    <<+>+>+>+ <<<< ]
H .
```

Listing 2: Semiclassical 5-qubit Shor algorithm in |BF>.

6.2 Recycled Operational Flow

The tape width is strictly compressed to $N = 5$ cells:

$$\text{Tape}_{\text{recycled}} = \Big[\underbrace{q_0}_{q_{\text{ctrl}}}, \underbrace{q_1}_{y_0}, \underbrace{q_2}_{y_1}, \underbrace{q_3}_{y_2}, \underbrace{q_4}_{y_3}\Big] \quad (14)$$

1. **LSB Phase Bit (ϕ_2):** Target is initialized to $|1\rangle = |1000\rangle$ via >+<. Control cell q_0 is placed in $|+\rangle$ via H. Controlled-U_4 produces the entangled state $\frac{1}{\sqrt{2}}(|0\rangle|1000\rangle + |1\rangle|0010\rangle)$. Applying H on q_0 and measuring (.) yields the LSB $\phi_2 \in \{0, 1\}$ and collapses the target to $\frac{1}{\sqrt{2}}(|1000\rangle + (-1)^{\phi_2}|0010\rangle)$.

2. **Zero-Ancilla Feed-Forward Stash:** Observe that after U_4, target cell q_4 (y_3) remains in $|0\rangle$ on both $|1000\rangle$ and $|0010\rangle$ (this holds only because the target starts in $|1\rangle$; it would fail for a general eigenstate input, whose support includes $|13\rangle = |1011\rangle$). Loop [»»+«« ,] temporarily stashes ϕ_2 onto q_4 and resets q_0 to $|0\rangle$.

3. **Pre-Commuted Phase Correction & MSB (ϕ_1):** Since diagonal phase gate $S_0^\dagger$ commutes with $\mathcal{C}_0(U_7)$, we apply H on q_0 and immediately execute »» [««-»» ,] ««. If $\phi_2 = 1$, this applies $S^\dagger$ ('-') to q_0 and actively resets $q_4 \to |0\rangle$ *prior* to Controlled-U_7. Controlled-U_7 then executes on the restored target register, followed by H and readout (.) to yield MSB ϕ_1. The classical transcript is therefore $\mathcal{C} = (\phi_2, \phi_1)$, i.e. in the *reverse* bit order of the full-register program.

This reduces spatial tape requirements by 16.7% and eliminates all two-qubit QFT entangling gates (the only remaining phase correction is a classically controlled single-qubit $S^\dagger$). Furthermore, since $y_1 = y_3 = 0$ on $|1\rangle$ and U_4 is applied first, SWAP(y_1, y_3) in U_4 acts trivially and can be omitted, reducing the number of native ^ tokens from 27 to 18.

6.3 Input-Aware Simplification and the "Compiled Shor" Caveat

The same argument applies to the full-register program: Controlled-U_4 is the first operation acting on the target register, which at that point is exactly $|1\rangle = |1000\rangle$, so SWAP(y_1, y_3) is the identity on every branch and can be dropped. This yields an equivalent 112-token program (examples/shor_15_full_opt.bfk) whose pre-measurement statevector coincides with that of Listing 1 to machine precision.

We stress the distinction from the "compiled" Shor demonstrations criticised by Smolin, Smith and Vargo [13]. Removing a gate that provably acts on qubits known to be $|0\rangle$ uses only knowledge of the *input* state, which any compiler may exploit. By contrast, replacing Controlled-U_4 by the two CNOTs that map $|1\rangle \leftrightarrow |4\rangle$, or truncating the multipliers to the orbit $\{1, 7, 4, 13\}$, presupposes knowledge of the period r—the very quantity the algorithm is meant to find—and would make the demonstration vacuous. All |BF> programs in this paper implement U_4 and U_7 as genuine modular multipliers on the full domain $\{1, \ldots, 14\}$ (Lemmas 1 and 2); the only input-aware step is the removal of SWAP(y_1, y_3) described above.

7 Transpilation to OpenQASM 3.0

Because pointer arithmetic (>, <) can be computed statically at compile time, pointer movement emits no gates. OpenQASM 3.0 gate modifiers such as ctrl @ apply only to named gates (there are no anonymous controlled blocks), so the automatic transpiler lowers every token of a Profile C block to its controlled counterpart (^→ccx, H→ch, +→cx, ...), assigns one transcript bit per . token, and uses separate guard bits for Profile D loop tests.

7.1 Conjugation-Aware Lowering of Profile C Blocks

Controlling every token of a Profile C block is correct but wasteful. The transpiler instead exploits the elementary identity

$$\mathcal{C}_p\Big(VUV^{-1}\Big) = V\,\mathcal{C}_p(U)\,V^{-1}, \quad (15)$$

Metric	|BF> (6-Qubit Full)	|BF> (6-Qubit Full, opt.)	|BF> (5-Qubit Recycled)	|BF> (5-Qubit Recycled, opt.)
Tape Register Width (N)	6	6	5	5
Source Instruction Tokens	149	112	169	132
Native ^ tokens	27	18	27	18
Multi-qubit gates, per-token lowering	27 ccx, 4 cx, 36 ch, 2 ctrl @ tdg	18 ccx, 4 cx, 24 ch, 2 ctrl @ tdg	27 ccx, 4 cx, 36 ch	18 ccx, 4 cx, 24 ch
CNOT-equivalents, per-token lowering	206	140	202	136
Multi-qubit gates, conjugation-aware lowering	5 ccx, 26 cx, 1 ctrl @ sdg	4 ccx, 18 cx, 1 ctrl @ sdg	5 ccx, 26 cx	4 ccx, 18 cx
CNOT-equivalents, conjugation-aware lowering	**58**	**44**	**56**	**42**
Circuit depth (ASAP), conjugation-aware	51	36	50	35
Max. interaction distance (all-to-all lowering)	5	5	4	4
Open linear chain (LNN): exact 1-/2-qubit decomposition + SWAP routing				
Routing SWAPs inserted	22	20	21	17
CNOT-equivalents incl. SWAPs (SWAP = 3)	**124**	**104**	**119**	**93**
Circuit depth (ASAP), LNN	120	92	121	93
Max. interaction distance, LNN	1	1	1	1
Measurements incl. loop tests / resets	2 / 0	2 / 0	6 / 2	6 / 2

Table 3: Resource analysis of the |BF> Shor programs ($N = 15$, $a = 7$), generated automatically from the reference transpiler by `tools/gen_paper_tables.py` (the test suite fails if this table is stale). CNOT-equivalents use standard decompositions: Toffoli = 6, controlled-phase (T, $T^\dagger$, S, $S^\dagger$) = 2, cx/cz/ch = 1. Per-token lowering controls every token of a Profile C block; conjugation-aware lowering (Sec. 7.1) controls only the gates that are not undone inside the block. The upper block assumes all-to-all connectivity: there, coherent controls couple c_0/c_1 (resp. q_0) to target cells up to five positions away. The lower block gives the cost on an open linear chain after exact decomposition into 1- and 2-qubit gates and SWAP routing (Sec. 7.2); every 2-qubit gate then acts on adjacent qubits. Loop-test measurements of the semiclassical programs are included in the measurement count.

valid for arbitrary unitaries U, V: if the control is $|0\rangle$, V and V^{-1} cancel. A dynamic program over the block's gate list chooses, for every contiguous segment, the cheaper of (i) splitting it into independently controlled parts and (ii) peeling off a prefix V whose inverse forms the matching suffix, leaving V uncontrolled; nested Profile C blocks are lowered first, and adjacent phase gates are merged ($T^\dagger T^\dagger \to S^\dagger$). For the SWAP macro S_k = `^(H>H<^>H<H)^` this applies twice and gives $\mathcal{C}(S_k) = \mathrm{CNOT}\,(H \otimes H)\,\mathrm{Toffoli}\,(H \otimes H)\,\mathrm{CNOT}$, i.e. 8 instead of 22 CNOT-equivalents. It also recognises that a distance-two transposition $S_j S_{j+1} S_j = V S_{j+1} V^{-1}$ with $V = S_j$ needs only its middle SWAP controlled. Together this reduces the 6-qubit program from 206 to 58 CNOT-equivalents (Table 3). Equivalence of the optimized and per-token lowerings with the abstract machine is checked exactly, by an independent OpenQASM interpreter, on all example programs and on several hundred randomly generated programs (differential fuzzing).

7.2 Routing onto an Open Linear Chain

To make the nearest-neighbor question measurable rather than rhetorical, the transpiler provides an LNN mode. First, every gate acting on three or more qubits is decomposed *exactly* into 1- and 2-qubit gates: the Toffoli by the standard Clifford+T circuit with six CNOTs [6]; multi-controlled phases recursively,

$$C^k P(\varphi) = \mathrm{CP}_{c_k t}(\tfrac{\varphi}{2})\, C^{k-1}X_{c_k}\, \mathrm{CP}_{c_k t}(-\tfrac{\varphi}{2}) \cdot C^{k-1}X_{c_k}\, C^{k-1}P(\tfrac{\varphi}{2}), \tag{16}$$

(written in time order, with $C^{k-1}X_{c_k}$ controlled by $c_1, \ldots, c_{k-1}$) together with $C^k X = H\, C^k P(\pi)\, H$, $C^k Z = H\, C^k X\, H$ and $C^k H = R_y(\tfrac{\pi}{4})\, C^k Z\, R_y(-\tfrac{\pi}{4})$ (operator products, i.e. $R_y(-\tfrac{\pi}{4})$ is applied first) [11]. Second, the resulting 2-qubit gates are routed on an open chain: a logical-to-physical layout is tracked and adjacent SWAPs are inserted greedily, choosing which endpoint to move by a short lookahead over the upcoming gates. Bodies of Profile D loops restore the layout they started with, so every iteration sees the same placement. Ring-wrapping ^ tokens at $p = N - 1$ are routed like any other non-adjacent gate.

For the 6-qubit program this yields 22 routing SWAPs and 124 CNOT-equivalents (including SWAPs at 3 CNOTs each), versus 58 with all-to-all connectivity; the optimized 5-qubit recycled program needs 17 SWAPs and 93 CNOT-equivalents (Table 3). Strict nearest-neighbor execution therefore costs roughly a factor of two over the all-to-all lowering. The router is a simple greedy heuristic, so these figures are upper bounds rather than optima. Exactness of every decomposition and of the routed circuits (including 4-controlled gates) is verified against the abstract machine by the independent interpreter and the differential fuzzer.

Listing 3 gives an equivalent, hand-structured

form of Listing 1 that wraps the modular multipliers in named gates:

```
OPENQASM 3.0;
include "stdgates.inc";

// Modular multiplication by 4 (SWAP
    network on y0..y3)
gate u4 y0, y1, y2, y3 {
    // Transposition (y0, y2)
    cx y0, y1; h y0; h y1; cx y0, y1; h y1
    ; h y0; cx y0, y1;
    cx y1, y2; h y1; h y2; cx y1, y2; h y2
    ; h y1; cx y1, y2;
    cx y0, y1; h y0; h y1; cx y0, y1; h y1
    ; h y0; cx y0, y1;
    // Transposition (y1, y3)
    cx y1, y2; h y1; h y2; cx y1, y2; h y2
    ; h y1; cx y1, y2;
    cx y2, y3; h y2; h y3; cx y2, y3; h y3
    ; h y2; cx y2, y3;
    cx y1, y2; h y1; h y2; cx y1, y2; h y2
    ; h y1; cx y1, y2;
}

// Modular multiplication by 7 (SWAP
    cascade + bit inversion)
gate u7 y0, y1, y2, y3 {
    cx y0, y1; h y0; h y1; cx y0, y1; h y1
    ; h y0; cx y0, y1;
    cx y1, y2; h y1; h y2; cx y1, y2; h y2
    ; h y1; cx y1, y2;
    cx y2, y3; h y2; h y3; cx y2, y3; h y3
    ; h y2; cx y2, y3;
    x y0; x y1; x y2; x y3;
}

qubit[6] q;
bit[2] c;

// Initialization
h q[0];
h q[1];
x q[2];

ctrl @ u4 q[1], q[2], q[3], q[4], q[5];
    // Controlled-U4 on c1
ctrl @ u7 q[0], q[2], q[3], q[4], q[5];
    // Controlled-U7 on c0

// 2-Qubit Inverse QFT on (q[0], q[1])
    without final SWAP
h q[1];
ctrl @ tdg q[1], q[0];
ctrl @ tdg q[1], q[0];
h q[0];

// Computational basis readout: c[0] =
    phi_1 (MSB), c[1] = phi_2 (LSB)
c[0] = measure q[0];
c[1] = measure q[1];
```

Listing 3: Hand-structured OpenQASM 3.0 circuit equivalent to the full-register |BF> Shor program.

8 NISQ Fidelity Estimate

On physical quantum hardware characterized by single-qubit error rate $\epsilon_1 \approx 10^{-4}$, two-qubit CNOT gate error $\epsilon_2 \approx 5 \times 10^{-3}$, and transverse dephasing time T_2, circuit survival fidelity is commonly modelled as:

$$\mathcal{F} \approx (1-\epsilon_1)^{G_1}(1-\epsilon_2)^{G_2} \exp\left(-\frac{D \cdot \tau_g}{T_2}\right) \qquad (17)$$

Counting only the two-qubit factor with the CNOT-equivalents of Table 3, the per-token lowering of the 6-qubit program ($G_2 = 206$) gives $(1-\epsilon_2)^{206} \approx 0.36$, whereas the conjugation-aware lowering ($G_2 = 58$) gives ≈ 0.75; the input-simplified 6-qubit program ($G_2 = 44$) and the optimized 5-qubit recycled program ($G_2 = 42$) reach ≈ 0.80 and ≈ 0.81 with all-to-all connectivity. On a strict open chain the routed circuits ($G_2 = 124$ and 93) give ≈ 0.54 and ≈ 0.63, respectively (decoherence not included). The dominant cost is the coherent control of the SWAP networks, not the native ^ primitive itself, which is why control-aware lowering matters more than any other optimization considered here.

9 Conclusion & Future Work

We have demonstrated that Shor's algorithm for $N = 15$, $a = 7$ can be compiled into the minimal |BF> calculus, with the modular multipliers expressed as nearest-neighbor SWAP networks and an exact phase readout, and that semiclassical dynamic loops reduce the register to five qubits without two-qubit QFT gates. The coherent control of these networks is not nearest-neighbor and dominates the gate cost after lowering; conjugation-aware lowering reduces it by a factor of about 3.5 (206 → 58 CNOT-equivalents for the 6-qubit program). Routed exactly onto an open linear chain, the programs remain executable with roughly twice the all-to-all cost. Future work will replace the greedy router by optimal SWAP placement, exploit relative-phase Toffolis where phases cancel, extend this framework to general modular exponentiation using linear reversible carry-ripple adders [7, 14], and pursue formal verification of the lowering within mechanized proof assistants.

References

[1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proc. 35th Ann. Symp. on Foundations of*

Computer Science (FOCS '94), pp. 124–134, IEEE, 1994, doi:10.1109/SFCS.1994.365700.

[2] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Review*, vol. 41, no. 2, pp. 303–332, 1999, doi:10.1137/S0036144598347011.

[3] mat-ocl, "|BF> (Bra-Ket-Fuck): A Vacuum-Initialized Clifford+T Quantum Tape Architecture for Linear Nearest-Neighbor Compilation and Formal Verification," *Technical Whitepaper v1.0*, Systems & Quantum Esoteric Architecture Working Group, Oct. 2026.

[4] U. Müller, "Brainfuck," 1993, online archive.

[5] P. A. M. Dirac, "A new notation for quantum mechanics," *Math. Proc. Cambridge Philos. Soc.*, vol. 35, no. 3, pp. 416–418, 1939, doi:10.1017/S0305004100021162.

[6] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary ed., Cambridge University Press, 2010, doi:10.1017/CBO9780511976667.

[7] A. G. Fowler, S. J. Devitt, and L. C. L. Hollenberg, "Implementation of Shor's algorithm on a linear nearest neighbour qubit array," *Quantum Info. Comput.*, vol. 4, no. 4, pp. 237–251, 2004, arXiv:quant-ph/0402196.

[8] M. Saeedi, R. Wille, and R. Drechsler, "Synthesis of quantum circuits for linear nearest neighbor architectures," *Quantum Information Processing*, vol. 10, no. 3, pp. 355–377, 2011, doi:10.1007/s11128-010-0201-2.

[9] A. Y. Kitaev, "Quantum measurements and the Abelian Stabilizer Problem," 1995, arXiv:quant-ph/9511026.

[10] R. B. Griffiths and C.-S. Niu, "Semiclassical Fourier transform for quantum computation," *Phys. Rev. Lett.*, vol. 76, no. 17, pp. 3228–3231, 1996, doi:10.1103/PhysRevLett.76.3228.

[11] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, "Elementary gates for quantum computation," *Phys. Rev. A*, vol. 52, no. 5, pp. 3457–3467, 1995, doi:10.1103/PhysRevA.52.3457.

[12] T. Monz, D. Nigg, E. A. Martinez, M. F. Brandl, P. Schindler, R. Rines, S. X. Wang, I. L. Chuang, and R. Blatt, "Realization of a scalable Shor algorithm," *Science*, vol. 351, no. 6277, pp. 1068–1070, 2016, doi:10.1126/science.aad9480.

[13] J. A. Smolin, G. Smith, and A. Vargo, "Oversimplifying quantum factoring," *Nature*, vol. 499, no. 7457, pp. 163–165, 2013, doi:10.1038/nature12290.

[14] S. Beauregard, "Circuit for Shor's algorithm using $2n+3$ qubits," *Quantum Info. Comput.*, vol. 3, no. 2, pp. 175–185, 2003, arXiv:quant-ph/0205095.